МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ ТА ІНФОРМАТИКИ

Кафедра
Інформаційних систем

АЛГОРИТМИ ОБЧИСЛЮВАЛЬНИХ ПРОЦЕСІВ

Звіт виконання лабораторної роботи №3

“Оптимізація алгебричних виразів”

Виконав:
Студент групи ПМі-12
Козій Д.Н.
Перевірили:
доц. кафедри ТОП Мельничин А. В.,
доц. кафедри ТОП Чипурко А. І.

Львів – 2026

Метод 1: Ітеративний підхід

Алгоритм:

1. Приймаємо змінні x та n.
2. Ініціалізуємо змінну S нулем.
3. Запускаємо цикл, де лічильник i йде від 1 до n включно.
4. На кожній ітерації додаємо до S значення sin(i*x).
5. Після завершення циклу повертаємо результат S.

Реалізація:

```
float calc1(int x, int n) {
    float S = 0;
    for (int i = 1; i <= n; i++) {
        S += sin(i * x);
    }
    return S;
}
```

Приклад роботи:

```
int main() {
    srand(time(NULL));
    int x, n = 100;
    auto start = chrono::high_resolution_clock::now();
    x = 175;
    calc1(x, n);
    auto time1 = chrono::duration_cast<chrono::nanoseconds>(chrono::hi
    cout << "x = " << x << ", time: " << time1 << "nanos." << endl;
```

Microsoft Visual Studio Debug

```
x = 175, time: 12700nanos.
```

Висновок:

Цей метод є найпростішим для розуміння та прямо відображає поставлене завдання. Проте він має складність O(n), оскільки використовує цикл. Основним недоліком є те, що для великих значень n програма виконує велику кількість ресурсомістких операцій виклику тригонометричної функції sin(), що робить цей підхід повільним та неефективним.

Метод 2: Тригонометрична формула

Алгоритм:

1. Приймаємо змінні x та n.
2. Перевіряємо базовий випадок: якщо x == 0, одразу повертаємо 0. Це дозволяє уникнути помилки ділення на нуль на наступних кроках.
3. Приводимо x до типу float і попередньо обчислюємо half_x , щоб не дублювати операцію ділення.
4. Обчислюємо результат за закритою математичною формулою:

$$S = \frac{sin\left(\frac{n \cdot x}{2}\right) sin\left(\frac{(n+1) \cdot x}{2}\right)}{sin\left(\frac{x}{2}\right)}$$

5. Для цього використовуємо швидшу версію функції синуса для чисел з плаваючою комою — sinf().
6. Повертаємо обчислене значення.

Реалізація:

```
float calc2(int x, int n) {
    if (x == 0) {
        return 0;
    }
    float fx = (float)x;
    float half_x = fx / 2.0f;

    return (sinf(n * half_x) * sinf((n + 1) * half_x)) / sinf(half_x);
}
```

Приклад роботи:

```
int main() {
    srand(time(NULL));
    int x, n = 100;
    auto start = chrono::high_resolution_clock::now();
    x = 175;
    calc2(x, n);
    auto time1 = chrono::duration_cast<chrono::nanoseconds>(chrono::high_resolution_clock::now
    cout << "x = " << x << ", time: " << time1 << "nanos." << endl;
```

Microsoft Visual Studio Debug

```
x = 175, time: 6400nanos.
```

Висновок:

Цей метод є математично оптимізованим і має складність O(1). Замість n ітерацій та викликів синуса, алгоритм зведено до єдиної формули, яка завжди вимагає лише трьох звернень до функції sinf(). Це дозволяє виконувати розрахунки миттєво, незалежно від розміру n, а також зменшує похибку накопичення при роботі з типом float.